THE ONLY PDF YOU NEED FOR

SYSTEM DESIGN INTERVIEWS.

Best resources. The exact approach. Core concepts. Top 10 questions. A study plan. Everything in one place — curated by someone who actually used all of it.

WHAT'S INSIDE

01 **The Resources** Best free and paid resources — videos, repos, and books.

02 **The Framework** Exact step-by-step approach for any SD question.

03 **Core Concepts** The 12 concepts that appear in every interview.

04 **Top 10 Questions** Most asked questions with what interviewers want to hear.

05 **Numbers Every Dev Knows** Latency, throughput, storage — memorise these.

06 **30-Day Study Plan** Exactly what to study, in what order, week by week.

Resources on page 1 are the exact ones maintained by Yatin Sharma — personally tested and still in use. Everything else is built around them.

@yatincodes

SECTION 01

The Resources

Curated by Yatin Sharma — not random picks. These are personally tested and still in use today.

FREE RESOURCES — START HERE

— **CodeJeet.com —** Chapter-wise system design with video walkthroughs and written notes. The best structured path for beginners and experienced engineers alike. Covers 16 full chapters from scale to millions of users to designing Google Drive. codejeet.com

— **Awesome System Design (GitHub) —** 38k+ stars. Covers scalability basics, distributed systems, databases, caching, load balancing. One of the most comprehensive free resources that exists. You'll come back to this repeatedly. github.com/ashishps1/awesome-system-design-resources

— **System Design Primer (GitHub) —** 270k+ stars. The most starred CS repo on GitHub. Covers every core concept with diagrams and real examples. github.com/donnemartin/system-design-primer

— **ByteByteGo (YouTube) —** 1.39M subscribers, 165 videos. Short visual animations that break down complex concepts in minutes. Watch before any interview round as a quick revision pass. youtube.com/@ByteByteGo

— **High Scalability (highscalability.com) —** Real architecture breakdowns of how companies like Twitter, Netflix, and Uber actually built their systems. Reading these is worth more than most courses.

WORTH PAYING FOR (IF YOU CAN)

— **Designing Data-Intensive Applications —** Martin Kleppmann. The definitive book on databases, distributed systems, and data pipelines. Read chapters 1-6 before any FAANG interview. Non-negotiable.

— **Grokking System Design (Educative.io) —** Interactive, problem-by-problem format. Covers 20+ real questions with guided solutions. Best for structured interview prep with a deadline.

— **ByteByteGo Newsletter —** Alex Xu (author of System Design Interview books). Weekly deep dives into real architectures. The books themselves are the clearest interview prep material available. blog.bytebytego.com

HOW TO USE THESE TOGETHER

Start with CodeJeet.com for structured concepts. Use the System Design Primer as your reference. Watch ByteByteGo to visualise anything that doesn't click. When you hit a question you can't solve, read how the real company solved it on highscalability.com. DDIA is your depth layer — read it in parallel, slowly.

SECTION 02

The Framework

Every system design question has the same shape. This is how you approach all of them.

THE 6-STEP APPROACH — USE THIS EVERY TIME

Interviewers don't expect a perfect answer. They want to see structured thinking. Follow these steps in order and you'll never freeze up or go off-track.

STEP 1 — Clarify Requirements (5 min) — Ask before designing. Functional: what does the system DO? Non-functional: scale, latency, availability, consistency needs. Never assume.

STEP 2 — Estimate Scale (3 min) — Daily active users. Reads vs writes per second. Storage needed per year. This tells you what kind of system you're designing. Small scale = simple. Large scale = distributed.

STEP 3 — Define the API (3 min) — Write out 2-3 core endpoints before drawing anything. POST /shorten, GET /:code, GET /:code/stats. This forces precision and shows you understand the interface.

STEP 4 — High-Level Design (10 min) — Boxes and arrows. Client, load balancer, API servers, DB, cache, CDN. Don't go deep yet. Show you understand the full picture first.

STEP 5 — Deep Dive (15 min) — Pick 2-3 components the interviewer cares about and go deep. How does the DB schema look? What caching strategy? How do you handle failures? This is where you prove real understanding.

STEP 6 — Trade-offs and Bottlenecks (4 min) — What would break first under load? What did you sacrifice? Strong vs eventual consistency? SQL vs NoSQL — and why? Naming your trade-offs is a senior signal.

THE GOLDEN RULE

Never jump to solutions before clarifying requirements.
The interviewer will always let you go deeper on what matters to them.
Saying 'I'd use a cache here and here's why' beats 'I'd use Redis'.
Think out loud. Silence is the only wrong answer.

@yatincodes

SECTION 03

Core Concepts You Must Know

Every system design question uses a subset of these 12. Know all of them.

Horizontal vs Vertical Scaling — Scale up (bigger machine) vs scale out (more machines). Know when each applies and why horizontal is preferred for web services.

Load Balancing — Distributes traffic across servers. Round robin, least connections, IP hash. Eliminates single point of failure.

Caching — Cache-aside, write-through, write-behind, refresh-ahead. Know where to cache (CDN, application layer, DB layer) and what happens when cache misses.

Database Sharding — Splitting data across multiple DB instances by a shard key. Enables horizontal DB scaling. Know hot key problems and how consistent hashing solves them.

SQL vs NoSQL — SQL: ACID, relationships, structured data. NoSQL: scale, flexibility, high write throughput. Know which to pick and why — not just the names.

Replication — Master-replica for read scale. Multi-master for write scale. Synchronous vs asynchronous — know the consistency trade-off of each.

CAP Theorem — Consistency, Availability, Partition tolerance — you can only guarantee two. CP systems (banks). AP systems (social feeds). Know what you're trading.

Message Queues — Decouple producers from consumers. Enable async processing, retry logic, and fan-out. Know at-least-once vs exactly-once delivery.

Consistent Hashing — Distributes load across nodes so adding/removing a node only remaps a fraction of keys. Used in distributed caches and load balancers.

CDN — Serves static assets from edge nodes close to the user. Reduces latency globally. Push vs pull CDNs. Know when to put dynamic content behind a CDN too.

Rate Limiting — Token bucket for burst-friendly limits. Sliding window for accurate limits. Per user, per IP, per API key. Know what breaks without it.

Consistent Reads — Eventual consistency: faster, scales better, some staleness. Strong consistency: always correct, more expensive. Know which your system can afford.

HOW TO STUDY THESE

Don't memorise definitions. For each concept, answer: what problem does it solve, what does it trade off, and name one system that uses it. That's the level of understanding an interview requires.

SECTION 04

Top 10 Most Asked Questions

These come up in over 80% of system design rounds. Know all 10 well.

01 Design a URL Shortener
Key topics: base62, key-value store, caching hot URLs, 301 vs 302 redirects, analytics pipeline.

02 Design Twitter / News Feed
Key topics: fan-out on write vs read, feed ranking, hot celebrity problem, eventual consistency.

03 Design WhatsApp / Chat
Key topics: WebSockets, message ordering, offline delivery via push, multi-device sync, E2E encryption overview.

04 Design YouTube / Netflix
Key topics: video upload pipeline, transcoding at scale, CDN for streaming, adaptive bitrate, metadata vs blob storage.

05 Design Uber / Location Service
Key topics: real-time location updates, geospatial indexing, driver matching algorithm, trip state machine.

06 Design Google Drive / Dropbox
Key topics: chunked uploads, delta sync, conflict resolution, block storage vs object storage, versioning.

07 Design a Rate Limiter
Key topics: token bucket vs sliding window, shared state across instances, atomic operations, fail-open vs fail-closed.

08 Design a Notification System
Key topics: fan-out queue, channel workers (push/email/SMS), idempotency keys, dead letter queue, priority lanes.

09 Design Search Autocomplete
Key topics: trie data structure, top-K results, caching suggestions, typeahead latency budget, personalization.

10 Design a Web Crawler
Key topics: BFS vs priority queue, politeness delays, deduplication, distributed frontier, rendering JS pages.

WHAT INTERVIEWERS ACTUALLY WANT TO HEAR

For every question above, the answer is never one thing. It's a conversation: requirements first, then scale estimates, then design, then trade-offs. The interviewer will push you deeper on one or two components. That depth is the actual interview — the high-level diagram is just the opening.

SECTION 05

Numbers Every Dev Must Know

Memorise these. Use them in estimates. They make you sound like someone who's built real things.

LATENCY NUMBERS (approximate)

L1 cache	**~1 ns**
L2 cache	**~4 ns**
RAM read	**~100 ns**
SSD read	**~100 μs**
HDD read	**~10 ms**
Network (same DC)	**~0.5 ms**
Network (cross-region)	**~150 ms**
Disk seek	**~10 ms**

STORAGE & THROUGHPUT

1 char	**1 byte**
1 image (compressed)	**~300 KB**
1 video (1080p/min)	**~150 MB**
1M users, 1 tweet/day	**~500 GB/year**
1 server, requests/sec	**~10K req/s**
1 MySQL server reads	**~10K QPS**
1 Redis node	**~100K QPS**
CDN throughput	**~1 TB/s**

HOW TO ESTIMATE IN AN INTERVIEW

01 Daily Active Users (DAU) × average actions/day = total requests/day.

02 Divide by 86,400 (seconds/day) for avg req/sec. Multiply by 3-5x for peak traffic.

03 Storage: avg object size × writes/day × 365 = yearly storage.

04 Bandwidth: avg request size × req/sec = bytes/sec throughput needed.

05 Always round aggressively. 86,400 ≈ 100,000. Interviewers want order of magnitude, not precision.

QUICK EXAMPLE — TWITTER SCALE

```
300M DAU. Each user reads 100 tweets/day, writes 1 tweet/day.
Read QPS: 300M × 100 / 86,400 ≈ 350,000 reads/sec (peak ~1M/s)
Write QPS: 300M × 1 / 86,400 ≈ 3,500 writes/sec
Storage: avg tweet 200 bytes. 300M writes/day × 200B = 60 GB/day.
Conclusion: heavily read-skewed. Cache is mandatory. Write path is manageable.
```

SECTION 06

30-Day Study Plan

Exactly what to study in what order. One concept a day. Three questions a week.

WEEK 1 — FOUNDATIONS

Day 1: Scaling — vertical vs horizontal. Read Primer Chapter 1.

Day 2: Load balancing — algorithms and health checks.

Day 3: Caching — strategies, eviction, TTL. Read about Redis.

Day 4: CDN — push vs pull, edge cases, dynamic content.

Day 5: Design URL Shortener end-to-end. Time yourself.

Day 6: Review + read one highscalability.com case study.

Day 7: Rest. Briefly revisit anything that didn't click.

WEEK 2 — DATA LAYER

Day 8: SQL vs NoSQL — when to use which and why.

Day 9: Database indexing — B-tree, composite, covering index.

Day 10: Sharding — shard keys, hot key problem, resharding.

Day 11: Replication — master-replica, async vs sync.

Day 12: Design Google Drive. Focus on the storage layer.

Day 13: DDIA Chapter 1-3. Slow read, take notes.

Day 14: Rest. Solve one question from memory.

WEEK 3 — DISTRIBUTED SYSTEMS

Day 15: CAP theorem — real examples of CP and AP systems.

Day 16: Message queues — Kafka concepts, fan-out, ordering.

Day 17: Consistent hashing — why and where it's used.

Day 18: Design Twitter feed. Focus on fan-out strategy.

Day 19: Design Notification System. Focus on reliability.

Day 20: Watch 5 ByteByteGo videos on topics above.

Day 21: Rest. Write down your 3 weakest concepts.

WEEK 4 — INTERVIEW PRACTICE

Day 22: Design WhatsApp. Focus on real-time + offline.

Day 23: Design YouTube. Focus on upload pipeline + CDN.

Day 24: Design Uber. Focus on location + matching.

Day 25: Design Rate Limiter. Focus on distributed state.

Day 26: Mock interview. Record yourself. Watch it back.

Day 27: Redo your 2 weakest questions from the week.

Day 28-30: One full mock per day. Framework every time.

THE ONLY RULE THAT MATTERS

Don't study passively. For every concept you read, immediately answer: what problem does it solve, what does it trade off, and where would you use it. For every question you study, close the solution and try to design it yourself first. The discomfort of not knowing is where the learning happens.

@yatincodes

YOU HAVE EVERYTHING YOU NEED.

Most people fail system design interviews not because they don't know enough — but because they panic, skip requirements, and start drawing boxes without thinking. The framework on page 2 fixes that. The concepts on page 3 give you the vocabulary. The questions on page 4 give you the practice.

YOUR STARTING POINT THIS WEEK

— **Day 1 —** Bookmark: codejeet.com, github.com/ashishps1/awesome-system-design-resources, github.com/donnemartin/system-design-primer.

— **Day 2 —** Read the framework on page 2 until you can recite all 6 steps without looking.

— **Day 3 —** Design a URL shortener using the framework. Write it out on paper. Don't look at solutions first.

— **Day 4 —** Watch 3 ByteByteGo videos on concepts you got wrong in your URL shortener design.

— **Day 5 —** Repeat with a new question. Pick one from the top 10 list.

FINAL THOUGHT

System design has no single right answer. Every decision is a trade-off. The engineer who gets hired is not the one who knows the most — it's the one who thinks clearly, communicates well, and explains why they made the choices they made. That skill is built by practice, not by reading.

@yatincodes